



UNIVERSITY OF
BATH

Modular Display Design and Implementation

Artem Dunaievskiy, Sanya Ahmad, Jake Briscoe, Jamie Turbyne
(ad2955 sa3581 jb3802 jt666)

EE12001: Design Skills

University of Bath

Word count: 3128

Date:

30 April 2025

Abstract

In this project, we designed and built a modular weather-display system that presents real-time weather information in an intuitive and aesthetically pleasing way. Our objectives were to create a working device that offered various features in a short period of time. The device uses a series of side-illuminated acrylic sheets, each engraved with weather symbols, which are selectively lit by LEDs to represent current conditions. An Arduino Nano ESP32 was used to connect to the Internet, retrieve weather data through an API, and control the display accordingly. The project combined circuit design, PCB fabrication, acrylic-sheet manufacturing, coding, and system integration. Testing showed that the device accurately displayed weather conditions, demonstrating the success of the design and implementation.

Table of Contents

Abstract	1
Table of Contents.....	2
List of Acronyms	3
1 Introduction.....	4
1.1 Context	4
1.2 Objectives	4
2 System Design.....	5
2.1 Initial Concept	5
2.2 Circuits design.....	5
2.3 PCB design	6
2.4 Display design	8
3 System Implementation	9
3.1 Acrylic sheets production	9
3.2 Assembly stage.....	9
3.3 Coding	9
3.4 Testing	10
4 Discussion	11
5 Conclusion.....	11
6 References	12
7 Appendices	13
7.1 Appendix A:	13
7.2 Appendix B	16
7.3 Appendix C	17
7.4 Appendix D	18

List of Acronyms

UoB	University of Bath
MOSFET	Metal–oxide–semiconductor field-effect transistor
LED	Light emitting diode
PCB	Printed circuit board
DRC	Design rules checker
CAD	Computer-aided design
SMD	Surface Mount Device
IoT	Internet of Things
JSON	JavaScript Object Notation
SSID	Service Set Identifier
API	Application Programming Interface

1 Introduction

1.1 Context

Weather plays a crucial role in our daily lives, often affecting our mood, choices and well-being. Whether it's remembering to bring a jacket on a cold day or applying sunscreen on a sunny one, staying informed about the weather can help us live happier and healthier lives.

With that in mind, we set out to design a product that could make weather data more accessible and intuitive for users. Our aim was to create a visually engaging and easy-to-use device that displays relevant weather information.

1.2 Objectives

To streamline the design process, we outlined several key objectives:

- Simple and intuitive interface
- Display weather information relevant to the user
- Information should be easy to understand immediately
- Automatic updates to weather information
- Convenient and reliable for daily use

2 System Design

2.1 Initial Concept

The aim of our project was to design and build a modular, side-illuminated display system that blended vintage aesthetics with modern electronics. Inspired by the “Lixie” display [1], our device used etched acrylic sheets illuminated by Light Emitting Diodes (LEDs) to represent different symbols. The primary function of the display is to present weather forecasts in a clear and engaging way. Each acrylic sheet represents a specific weather condition (e.g., sunny, cloudy, rain), and by selectively lighting up certain sheets, the display conveys dynamic weather information. This approach provided both an intuitive user interface and a visually distinctive design.

2.2 Circuits design

The first steps to development were design the control circuits, focusing particularly on LED control. A key component in our circuit is the Metal Oxide-semiconductor field-effect transistor (MOSFET), which allows us to control high- power LEDs using low-current signals from an Arduino pin.

Summarising the lab script [1], the circuit is centred around the use of a MOSFET, which serves as a crucial component for enabling effective switching of high-power LEDs. The MOSFET allows an Arduino pin, which typically supplies a low-current control signal, to manage significantly higher power loads. This was particularly useful when working with power-hungry components like LEDs, as it prevents overloading the Arduino while maintaining full control over the circuit behaviour.

As illustrated in Figure 1, the circuit also incorporates a pull-down resistor connected to the MOSFET gate. This resistor plays an important role in preventing unwanted or floating signals, especially during the Arduino board’s boot-up sequence when pin states may be undefined. Additional resistors are used throughout the circuit to limit current, ensuring that each component, particularly the LEDs, receive an appropriate and safe level of voltage and current.

In the subsequent step, a switch was integrated into the design to serves as a user-interface element. This allows users to manually interact with the circuit and potentially add more features. The switch functions using a common method in which the voltage drops when the button is pressed. Like the MOSFET control pin, the switch also includes a pull-down resistor for protective purposes.

To further enhance the reliability and stability of the circuit, decoupling capacitors were added to the power supply lines. These capacitors help smooth out any voltage fluctuations and protect components sudden transients. Additionally, practical aspects such as Arduino

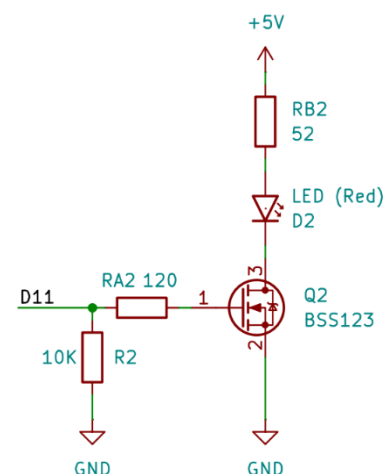


Figure 1 LED control circuit

connection headers and Printed Circuit Board (PCB) mounting holes were taken into consideration during the design process as well.

All schematic work was completed using KiCad Schematic Editor, a powerful Computer Aided Design (CAD) tool which streamlines the electronic design process. Additionally, it ensured smooth transition from component placement and wiring into the PCB layout. The finalized schematic produced is included in Appendix C for reference.

2.3 PCB design

Following the circuit design stage, the project smoothly transitioned into PCB design phase. This stage was carried out using the KiCad software, which, as previously mentioned, offers a comprehensive and user-friendly environment for designing printed circuit boards. Since this project is supported by the Department of Electronic Engineering, all required components were made available in advance. A detailed Bill of Materials (BoM), listing all components along with their specifications, is provided in Appendix D for reference.

As outlined earlier in the Project Week Report [2], several important constraints began to shape the PCB design. These constraints includes both general design rules and project specific conditions. Standard limitations such as surface-mount devices (SMD) placement, clearance and strategic positioning of high-visibility elements like LEDs had to be addressed. Additionally, constraints imposed by the manufacturing process—including board dimensions, minimum trace widths—had to be considered. These considerations were crucial in ensuring that the design would be both functional and manufacturable, aligning with real-world production standards.

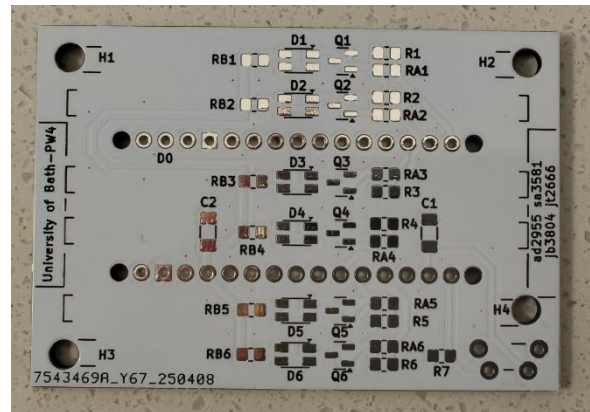


Figure 2 Produced PCB

Once the preparatory steps were completed, the actual PCB layout work could begin. This process started with generating mechanical drawings and assigning schematic placements for each component. Careful attention was given to the inclusion of dedicated mounting holes and reserved space for user-interface elements such as displays. Since the final product is housed within a provided display enclosure, all mechanical dimensions were verified to ensure it all fit properly.

Component placement followed, with all SMD components strategically positioned on the top layer of the PCB. This decision was made to facilitate a more streamlined manufacturing process, particularly when employing automated reflow soldering methods.

With the components placed, the routing phase commenced. Routing was carried out with attention to both the functional connections, based on the selected Arduino pins, and the physical constraints of component layout. All signal and power traces were successfully routed on a single copper layer, which simplified the board design while maintaining electrical performance. Special care was taken in defining trace widths: for instance, the 5V power supply line was routed with a width of 0.7 mm to accommodate higher current flow and reduce voltage drops across the board.

2.4 Display design

To visually relay the weather information to the user, a side-illuminated display system was implemented. This design utilised the series of LEDs mounted on the PCB to light up parallel acrylic sheets placed directly above them. Each sheet acts as an individual display layer. When powered, the light travels through the acrylic and reveals the engraved symbols, which are etched onto each sheet.

Referencing the lab script [3], Each acrylic sheet features a unique weather-related symbol; these symbols activate individually or in combination depending on the weather data received by the system. The positioning of the symbols was deliberately planned so that they could overlap visually. This overlapping permit more complex weather states, such as displaying both the “sunny” and “cloudy” symbols simultaneously to represent partially cloudy conditions. This layered approach gives the display a dynamic quality and visually emulates the style of traditional Nixie-tube displays.

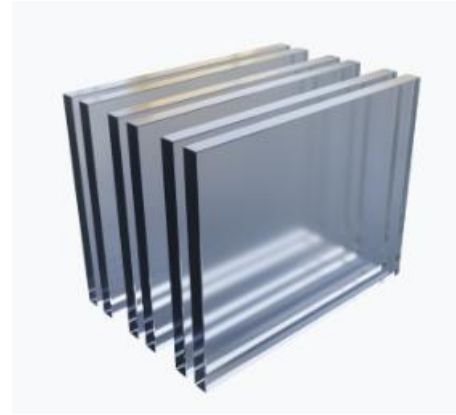


Figure 3 Acrylic Layers [3]

This setup not only provides a functional and intuitive method for displaying weather data but also adds an aesthetic layer to the product. The engraved icons are simple and easily interpretable briefly, while the glowing effect gives the system a sleek and modern appearance. The symbols were designed using Autodesk Inventor’s built-in engineering-drawing tools and rendered with a laser cutter, ensuring both precision and consistency.

3 System Implementation

3.1 Acrylic sheets production

Our display features six parallel acrylic sheets, each engraved with a unique symbol representing key weather characteristics: sun (yellow LED), wind (white LED), clouds (white LED), ice (blue LED), and happy (green LED) or sad (red LED) emojis to indicate overall good or bad conditions. Each sheet is illuminated by colour-matched LEDs and measure 59.3 x 39.3 x 3 mm to fit within the product enclosure. The final designs were created in Autodesk Inventor and are shown in [Appendix D]. As previously mentioned, once complete, the sheets were laser cut and prepared for integration into the device.

3.2 Assembly stage

To assemble the complete PCB, we used both surface mount and through-hole soldering. We began with the surface mount components. Using a stencil, we covered the PCB, leaving only the exposed pads where components were meant to be attached. A thin coat of solder paste was applied over the stencil, and any excess was removed before peeling the stencil away. Each surface mount component - LEDs, capacitors, and resistors - was then placed onto the solder paste in the correct position. After placing all the components, the PCB was placed in a reflow oven to melt the solder paste and fixed the components onto the board.

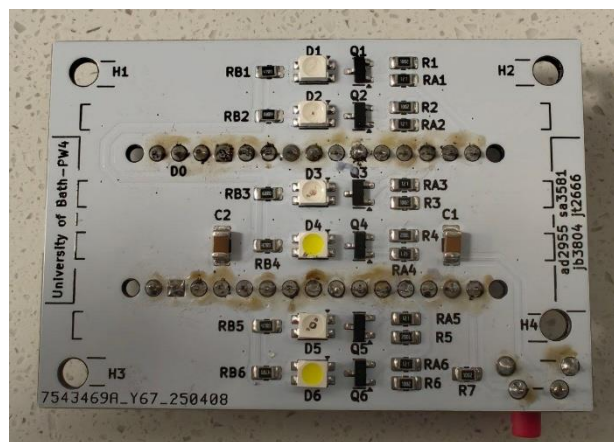


Figure 4 Assembled PCB

Once the PCB had cooled and the surface mount components were securely attached, we moved to through-hole soldering. We inserted the switch component and Arduino headers into their designated slots, making sure to clip the headers to fit the holes in the PCB. Using a coil of solder wire, we carefully melted solder into each hole, securing the switch and headers firmly in place.

3.3 Coding

To create the Modular Display design capable of showing accurate weather information, we used an Arduino Nano ESP32 [4], programmed in the Arduino language with the code shown in [Appendix A]. This microcontroller was essential due to its Internet of Things (IoT) capabilities, which allowed us to extract real-time data from reputable weather sources, control the LEDs programmatically, and implement algorithms to interpret the incoming data.

Before retrieving weather data, we first connected the Arduino to the Bath-IoT network using the provided Service Set Identifier (SSID) and password. This required including a Wi-Fi library in the preprocessor directives, as well as an HTTP Client library to facilitate communication with web services like OpenWeatherMap [5].

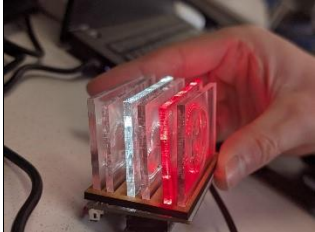
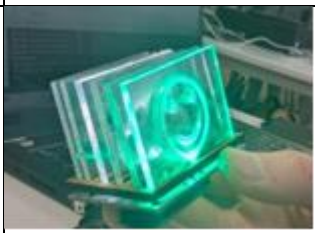
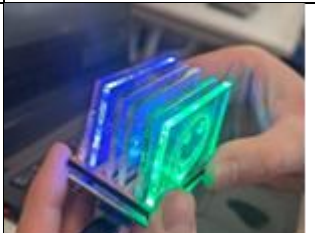
We then set up an Application Programming Interface (API) key, which functions like a personal ID granting access to weather service and allowing up to 60 requests per hour. To comply with this limit, we used a 1-minute delay between requests. The weather data was returned in JavaScript Object Notation (JSON) [6] format, which we parsed into a dictionary so we could extract specific values.

With the weather data stored in a dictionary, we extracted the icon code, which represents the current weather condition for the selected location. This icon code was used to determine which acrylic symbol to illuminate. Using the lookup table provided in the project week lab notes, we matched each icon code to its corresponding LED, allowing us to activate the appropriate display elements.

3.4 Testing

To test the functionality of our device, we input weather data from various regions around the world and recorded the system's output. We also tested its real-time functionality by connecting it to the internet and checking the weather for Bath, which is shown in the table below.

Table 1 Results of Testing

City	Expected output	Output received	Photo of output
Mumbai	Sunny	Sunny	
Moscow	Bad weather, cloudy	Bad weather, cloudy	
Bath	Good weather, cloudy	Good weather, cloudy	
Tromsø	Snowy	Snowy	

In all cases, the serial monitor displayed the expected results, and the correct LEDs were activated based on the weather data. This confirmed that the device could accurately update to different locations, interpret current weather information, and control the display as intended.

4 Discussion

Throughout the project, we encountered and overcame a range of design and implementation challenges. One of these challenges was ensuring that the PCB layout was compact yet functional, particularly given the constraints of using both surface-mount and through-hole components. Careful planning had to be done during the schematic and PCB design phase to ensure the final board fit the required enclosure dimensions and could be assembled efficiently.

Another major aspect was coding the Arduino to reliably connect to Wi-Fi and extract live weather data from various websites. The limitations on API request rates required us to implement appropriate timing delays to avoid exceeding any limits, ensuring the device's consistent performance.

A major strength of the final product was the smooth integration between different technical elements: electronics, mechanical design and software. Each part of the system complemented the others, leading to a cohesive and polished final device.

While the device performed well in testing, a few areas could be improved for future development. For example, expanding the number of weather conditions represented could improve the display's versatility, and refining the acrylic sheet design could further enhance visibility and aesthetics.

5 Conclusion

Overall, the project successfully met its objectives: creating a simple, intuitive, and aesthetically pleasing modular display system for presenting live weather information. The use of side-illuminated acrylic sheets provided an effective method for communicating weather conditions. The device reliably connected to the internet, retrieved weather data, and updated its display appropriately, as confirmed through a series of tests. Moving forward, small improvements to the symbol library, user interface, and overall design could further enhance the device's functionality.

6 References

- [1] L. Jabban and J. Graham-Harper-Cater, 'Modular Display Project - PCB Schematic - EE12001'. Accessed: Apr. 26, 2025. [Online]. Available: https://moodle.bath.ac.uk/pluginfile.php/2606566/mod_resource/content/31/_site/PCB-Assembly-and-Design/301-modular-display-project-pcb-schematic.html
- [2] A. Dunaievskyi, 'Project Week 4 - Modular Display Device - University of Bath Mahara'. Accessed: Apr. 27, 2025. [Online]. Available: <https://mahara.bath.ac.uk/view/blocks.php?id=74237>
- [3] L. Jabban and Graham-Harper-Cater, 'General Information - Year 1 Project Weeks'. Accessed: Apr. 30, 2025. [Online]. Available: https://people.bath.ac.uk/lj386/PW/PW4/general_info.html
- [4] 'Nano ESP32 | Arduino Documentation'. Accessed: Apr. 27, 2025. [Online]. Available: <https://docs.arduino.cc/hardware/nano-esp32/>
- [5] 'Current weather and forecast - OpenWeatherMap'. Accessed: Apr. 27, 2025. [Online]. Available: <https://openweathermap.org/city/2643743>
- [6] 'JSON'. Accessed: Apr. 30, 2025. [Online]. Available: <https://www.json.org/json-en.html>

7 Appendices

7.1 Appendix A:

Arduino Code for the weather Trixie light.

```
//Weather Trixie Display by jb3804, jt2666, ad2955, sa3581
//Libraries which allow me to use built in
#include <arduino.h>
#include <Wire.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <Arduino_JSON.h>

//Defining pins to make it more readable
const int greenLED = 12;    // LED 1 - Happy Face - Good Weather
const int redLED = 11;     // LED 2 - Sad Face - Bad Weather
const int yellowLED = 10;  // LED 3 - Sun - Sunny Weather
const int whiteLED1 = A2;  // LED 4 - Cloud - Cloudy Weather
const int blueLED = A1;    // LED 5 - Snowflake - Snowy Weather
const int whiteLED2 = A0;  // LED 6 - Wind - Windy Weather

//The SSID and password given to connect to UoB internet
const char* ssid = "UoB-IoT";
const char* password = "5dpt43v3";

//Makes these global variables to be accessed outside of the function
String weatherIcon = "";
String weatherMain = "";
int weatherID = 0;

void setup() {
    //Initialising the GPIO pins to activate the LEDs
    pinMode(greenLED, OUTPUT);
    pinMode(redLED, OUTPUT);
    pinMode(yellowLED, OUTPUT);
    pinMode(whiteLED1, OUTPUT);
    pinMode(blueLED, OUTPUT);
    pinMode(whiteLED2, OUTPUT);

    Serial.begin(115200);
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
```

```

        delay(500);
        Serial.print(".");
    }
    Serial.println("");
    Serial.println("WiFi connected.");
}

void getJSON(const char* url) {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin(url);
        int httpCode = http.GET();

        if (httpCode > 0) {
            String payload = http.getString();
            Serial.println(httpCode);
            Serial.println(payload);

            JSONVar data = JSON.parse(payload);
            weatherIcon = (const char*)data["weather"][0]["icon"];
            weatherMain = (const char*)data["weather"][0]["main"];
            weatherID = (int)data["weather"][0]["id"];

        } else {
            Serial.println("Error on HTTP request");
        }
        http.end();
    } else {
        Serial.println("Not connected to WiFi");
    }
}

// Then light up based on the icon
void handleWeatherLEDs() {
    // Turn off all LEDs
    digitalWrite(greenLED, LOW);
    digitalWrite(redLED, LOW);
    digitalWrite(yellowLED, LOW);
    digitalWrite(whiteLED1, LOW);
    digitalWrite(blueLED, LOW);
    digitalWrite(whiteLED2, LOW);

    // Show LEDs based on weather ID code

    // Decide sad or happy face based on icon

```

```

    if (weatherIcon.startsWith("09") || weatherIcon.startsWith("10") ||
weatherIcon.startsWith("11")) {
        digitalWrite(redLED, HIGH); // Rain / Thunderstorm = Sad Face
    } else {
        digitalWrite(greenLED, HIGH); // Otherwise = Happy Face
    }

    // Now light up secondary LEDs
    if (weatherIcon.startsWith("01")) {
        digitalWrite(yellowLED, HIGH); // Clear sky = Sun Acrylic
    }

    if (weatherIcon.startsWith("02") || weatherIcon.startsWith("03") ||
weatherIcon.startsWith("04")) {
        digitalWrite(whiteLED1, HIGH); // Cloudy conditions = Cloud Acrylic
    }

    if (weatherIcon.startsWith("13")) {
        digitalWrite(blueLED, HIGH); // Snow = Snow Acrylic
    }

    if (weatherIcon.startsWith("50")) {
        digitalWrite(whiteLED2, HIGH); //Mist = Wind Acrylic
    }

}

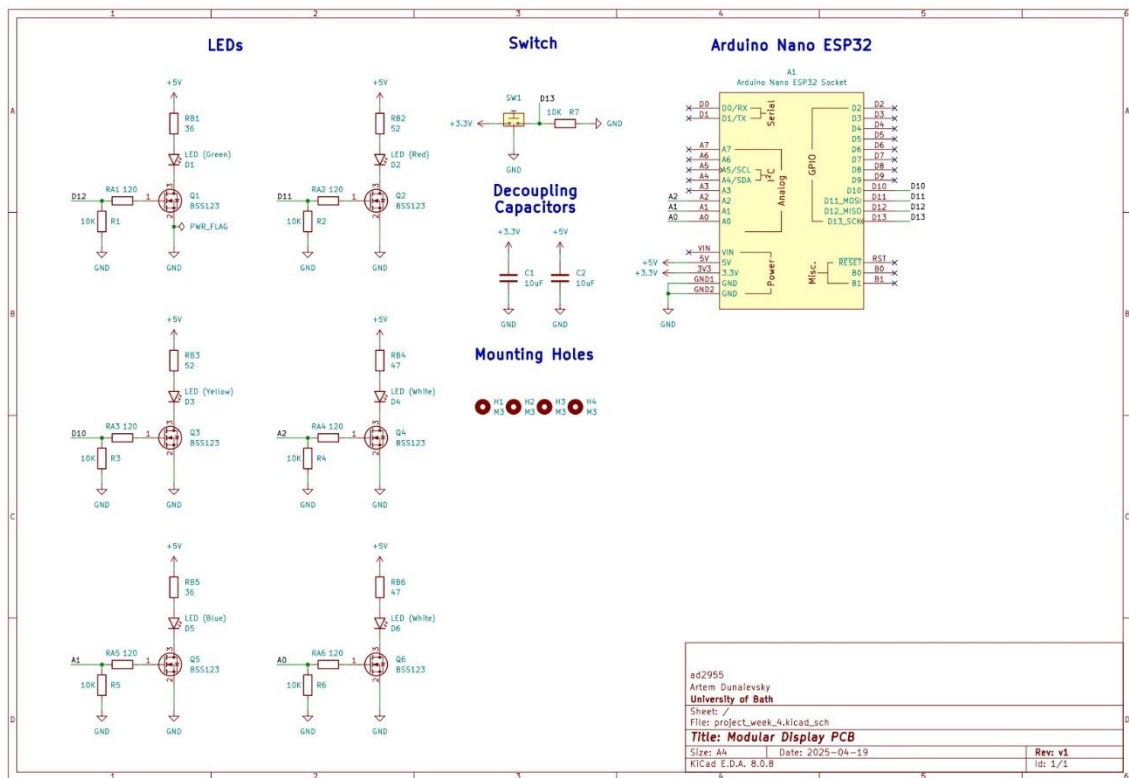
void loop() {
    //Replace q= with the city and the country code to change location
    //getJSON extracts and stores the information in the global variable

    getJSON("http://api.openweathermap.org/data/2.5/weather?q=Mawsynram,IN&appid=e
99290df7b2b596c342498ae42427d4e");
    //Prints the information in serial monitor for debugging
    Serial.println(weatherMain);
    Serial.print("Main: ");
    Serial.print("ID: ");
    Serial.println(weatherID);
    //Uses the information stored in global variables to set LEDs
    handleWeatherLEDs();
    delay(60000); // Wait 1 minute before checking again because of limit
}

```

7.2 Appendix B

Schematic of PCB:



7.3 Appendix C

Bill of Materials:

N	References	Value	Footprint	Description	Farnell Part Number	Manufacturer	Price/Unit [£]	Quantity
1	C1, C2	10uF	C_1206_3216Metric_Pad 1.33x1.80mm_HandSolder	Unpolarized capacitor	3881062	MULTICOMP PRO	2.07	2
2	R1, R2, R3, R4, R5, R6, R7	10K	R_0805_2012Metric_Pad 1.20x1.40mm_HandSolder	Resistor	2447553	MULTICOMP PRO	0.08	7
3	D4, D6	LED (White)	PLCC-4_3.2x2.8mm_WURTH_ HandSolder	LED (White)	1649009	CREE LED	0.87	2
4	D1	LED (Green)	PLCC-4_3.2x2.8mm_WURTH_ HandSolder	LED (Green)	3619644	WURTH ELEKTRONIK	0.32	1
5	D2	LED (Red)	PLCC-4_3.2x2.8mm_WURTH_ HandSolder	LED (Red)	3619649	WURTH ELEKTRONIK	0.28	1
6	D3	LED (Yellow)	PLCC-4_3.2x2.8mm_WURTH_ HandSolder	LED (Yellow)	3619650	WURTH ELEKTRONIK	0.32	1
7	D5	LED (Blue)	PLCC-4_3.2x2.8mm_WURTH_ HandSolder	LED (Blue)	3619646	WURTH ELEKTRONIK	0.32	1
8	SW1	~	SW-PHAP5-30RA	Tactile switch	3796560	APEM	1.55	1
9	A1	Arduino Nano ESP32 Socket	Arduino_Nano_ESP32	Arduino Nano ESP32 Socket	2212S-24SG-85	MULTICOMP PRO	1.52	1
10	Q1, Q2, Q3, Q4, Q5, Q6	BSS123	SOT-23	0.17A Id, 100V Vds, N-Channel MOSFET, SOT-23	1510764	NEXPERIA	0.99	6
11	RA1, RA2, RA3, RA4, RA5, RA6	120	R_0805_2012Metric_Pad 1.20x1.40mm_HandSolder	Resistor	2073628	MULTICOMP PRO	0.12	6
12	RB1, RB5	36	R_0805_2012Metric_Pad 1.20x1.40mm_HandSolder	Resistor	2447508	MULTICOMP PRO	0.14	2
13	RB2, RB3	52	R_0805_2012Metric_Pad 1.20x1.40mm_HandSolder	Resistor	2694892	MULTICOMP PRO	0.04	2
14	RB4, RB6	47	R_0805_2012Metric_Pad 1.20x1.40mm_HandSolder	Resistor	2670099	MULTICOMP PRO	0.16	2

7.4 Appendix D

Engineering drawing of acrylic laser design

